

4.5 Detecting ranges using logical operators

Settings

Beta features

logical operator

A logical operator treats operands as being True or False, and evaluates the result as True or False. Logical operators include AND, OR, and NOT.

PARTICIPATION
ACTIVITY

4.5.1: Logical operators: AND, OR, and NOT.

Start

☐ 2x speed

a	b	a AND b
False	False	False
False	True	False
True	False	False
True	True	True

a	b	a OR b
False	False	False
False	True	True
True	False	True
True	True	True

a
False
True

Let x = 7, y = 9

(x > 0) AND (y < 10) True
True True

(x < 0) OR (y > 10) False
False False

NOT (x < 0)
False

(x > 0) AND (y < 5) False
True False

(x < 0) OR (y > 5) True
False True

NOT (x > 0)
True

Captions 

1. AND evaluates to True only if BOTH operands are True.
2. OR evaluates to True if ANY operand is True (one, the other, or both).
3. NOT evaluates to the opposite of the operand.
4. Each operand is an expression. If $x = 7$, $y = 9$, then $(x > 0)$ AND $(y < 10)$ is True AND True, so True (both operands are True).

Table 4.5.1: Logical operators.

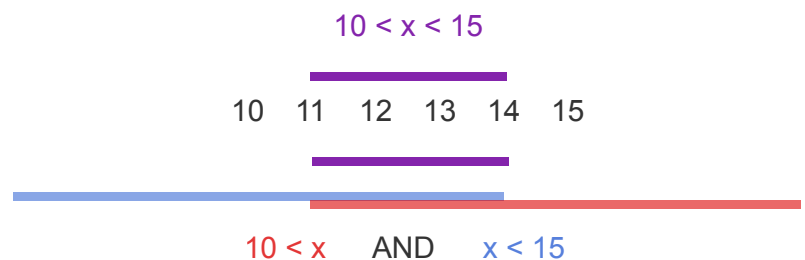
Logical operator	Description
a AND b	Logical AND: True when both of its operands are True
a OR b	Logical OR: True when at least one of its two operands is True
NOT a	Logical NOT: True when its one operand is False, and False when its operand is True

**PARTICIPATION
ACTIVITY**

4.5.3: Using AND to detect if a value is within a range.

Start

2x speed



Captions ^

1. The range $10 < x < 15$ means that x may be 11, 12, 13, or 14.
2. Specifying that range in a program can be done using two $<$ operators along with an AND operator and higher.
3. $x < 15$ defines the range 14 and lower. ANDing yields the overlapping range. Only when x is in the overlapping range are both expressions true.

Boolean

A Boolean refers to a value that is either True or False.

Figure 4.5.1: Creating a Boolean.

```
my_bool = True    # Assigns my_bool with the boolean value True
is_small = 4 < 3  # Assigns is_small with the result of the expression 4 < 3 (False)
```

and / or / not

Keywords and, or, and not (lowercase) are used to represent the AND, OR, and NOT operators.

Table 4.5.2: Logical operators.

Logical operator	Description
a and b	Boolean AND: True when both operands are True.
a or b	Boolean OR: True when at least one operand is True.
not a	Boolean NOT (opposite): True when the single operand is False (and False when the operand is True).

Table 4.5.3: Logical operators examples.

Given age = 19, days = 7, user_char = "q"

<code>(age > 16) and (age < 25)</code>	True, because both operands are True.
<code>(age > 16) and (days > 10)</code>	False, because both operands are not True.
<code>(age > 16) or (days > 10)</code>	True, because at least one operand is True.
<code>not (days > 10)</code>	True, because operand is False.
<code>not (age > 16)</code>	False, because operand is True.
<code>not (user_char == "q")</code>	False, because operand is True.

Figure 4.5.2: Detecting ranges: Cable TV channels.

```
if (user_channel >= 2) and (user_channel <= 499):  
    channel_type = "s"  
  
elif (user_channel >= 1002) and (user_channel <= 1499):  
    channel_type = "h"  
  
else:  
    channel_type = "e"
```

Try 4.5.1: Detecting ranges: Cable TV channels.

Run the program, enter 3 as input, and observe the output. Run the program again with another r

**PARTICIPATION
ACTIVITY**

4.5.8: Detecting ranges implicitly vs. explicitly.

Start ☐ 2x speed

```
if x < 0 :  
    # Negative
```

```
if x < 0 :  
    # Negative
```

```

elif (x >= 0) and (x <= 10) :
    # 0..10

elif (x >= 11) and (x <= 20) :
    # 11..20

else :
    # 21+

```

Explicitly defined ranges

```

elif (x <= 10) :
    # 0..10

elif (x <= 20) :
    # 11..20

else :
    # 21+

```

Implicitly defined ranges

Captions ^

1. This code detects ranges explicitly using the AND operator. The first branch executes when $(x \geq 0)$ and $(x \leq 10)$.
2. If the first branch doesn't execute, x must be ≥ 0 . So the second branch's expression is $x < 11$.
3. Implicit ranges can simplify a multi-branch if the statement for ranges is without gaps.

CHALLENGE ACTIVITY

4.5.1: Detecting ranges using logical operators.

763650.1722066.qx3zqy7

Start

Modify the given if statement so that 'Large town' is output if num_residents is in the range 1100 to 4600.

[Click here for examples](#) ▼

```
1 num_residents = int(input())
2
3 # Modify the following line
4 if (num_residents >= 1100) or (num_residents <= 4600):
5     print("Large town")
6 else:
7     print("Not a large town")
```

1

2

3

Check**Next level**

View solution ▼ *(Instructors only)*